Statistical Code Review Report: CLBoost

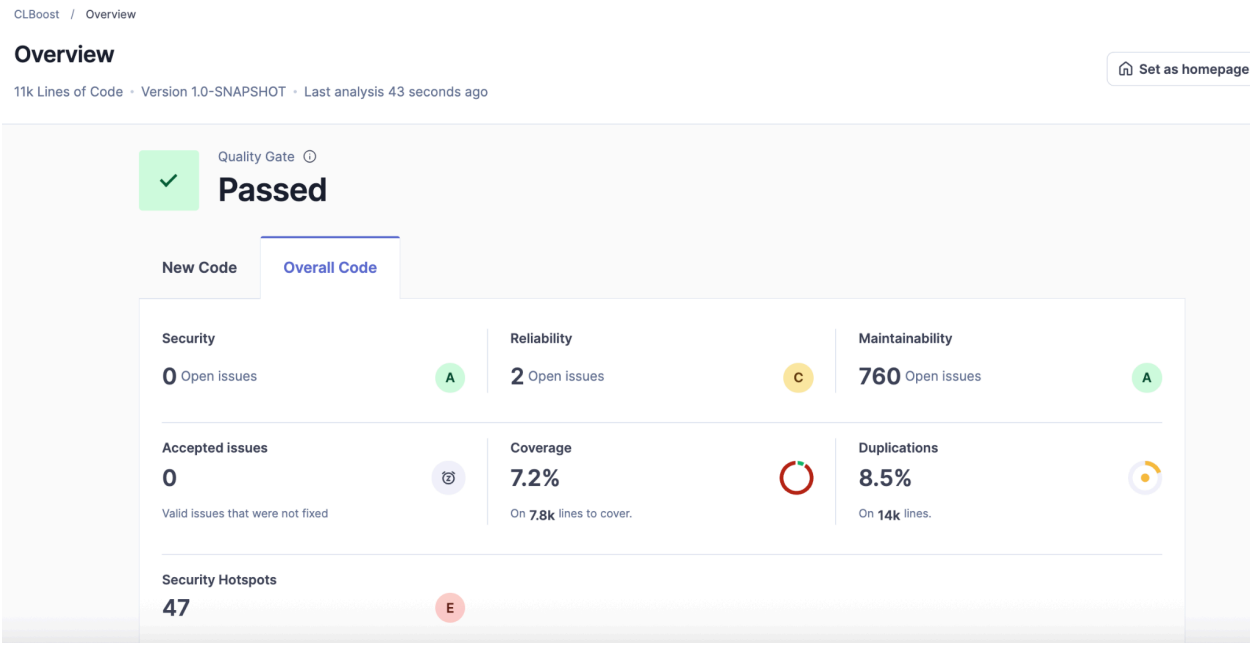
Tooling: SonarQube Community Build / SonarScanner

Date: April 10, 2026

1. Summary

The static analysis for the CLBoost project shows a passed status for the overall Quality Gate. However, while the Maintainability is rated an A, several underlying metrics suggest significant technical debt. The most concerning areas are the low unit test coverage (7.2%) and a high number of security hotspots that need manual review.

2. Summary Metrics

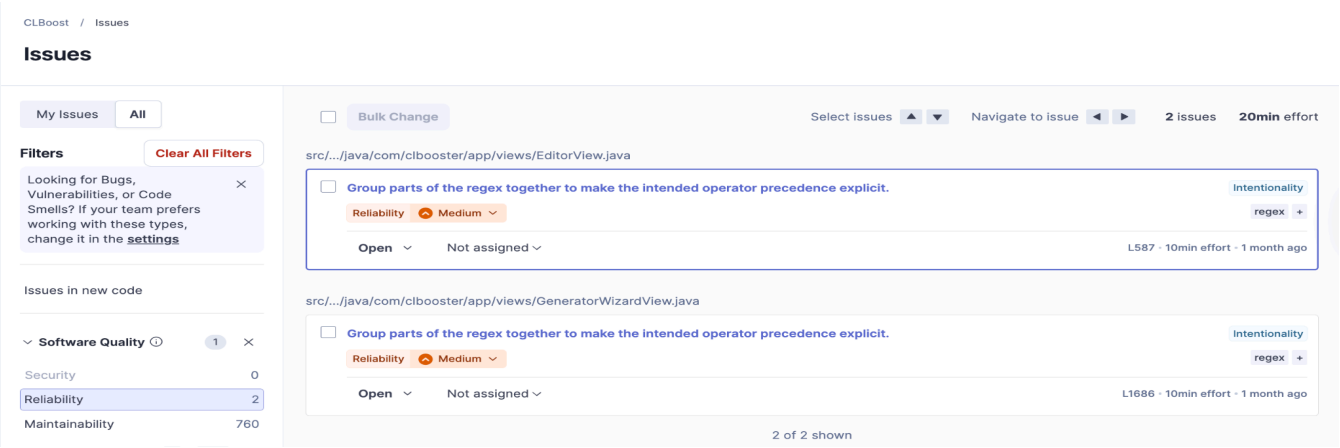


Metric	Value	Rating
Lines of Code (LOC)	~11,000	N/A
Security Issues	0 Open	A
Reliability Issues	2 Open	**C**
Maintainability Issues	760 Open	A
Security Hotspots	47	**E**

Code Coverage	7.2%	**F**
Duplications	8.5%	Warning

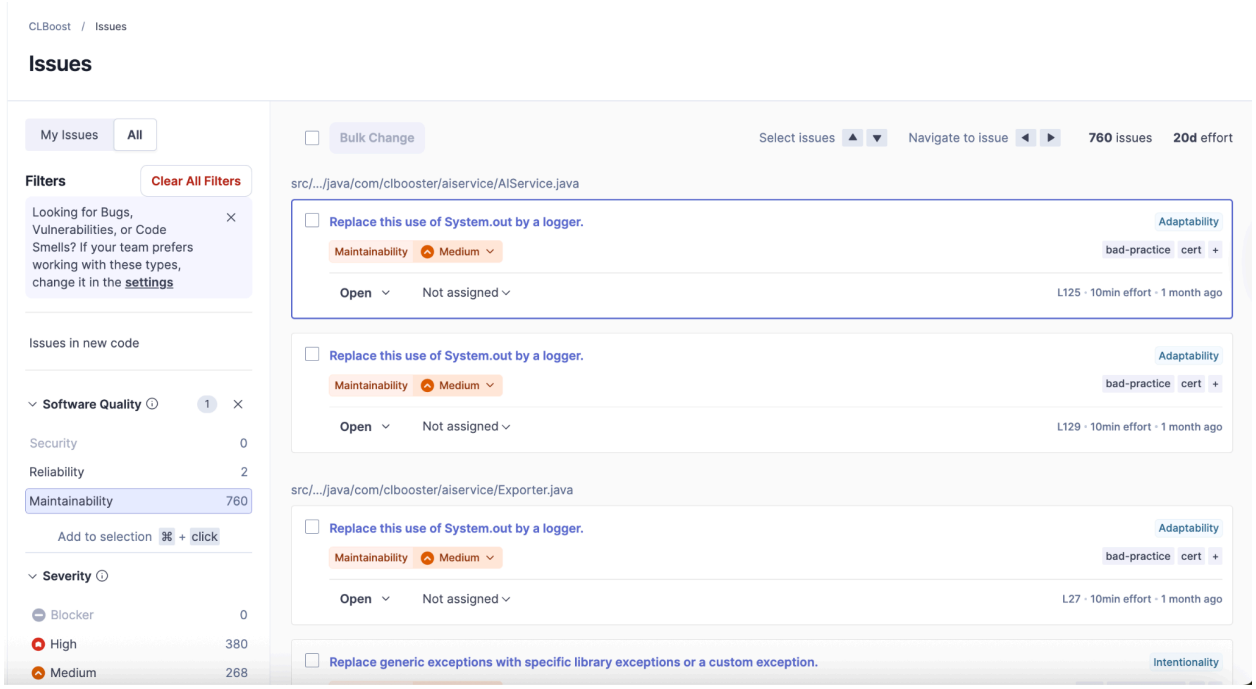
3. Key Findings & Analysis

3.1. Reliability and Code Correctness



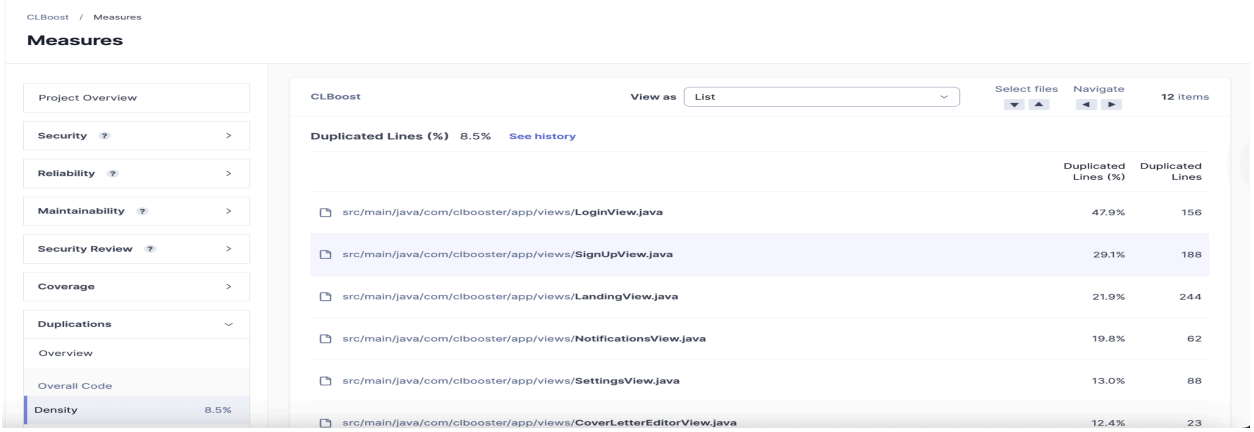
During the scan, we found two major reliability issues in EditorView.java and GeneratorWizardView.java classes involving the use of Regular Expressions (Regex). The tool flagged these because the regex logic is written in an ambiguous way, making it unclear how the code should prioritize different symbols. This is dangerous because it can cause the app to behave unexpectedly or lead to a "ReDoS" (Regular Expression Denial of Service) attack, where a user could crash the system just by entering a specific string of text that the messy regex can't handle.

3.2. Maintainability & Technical Debt



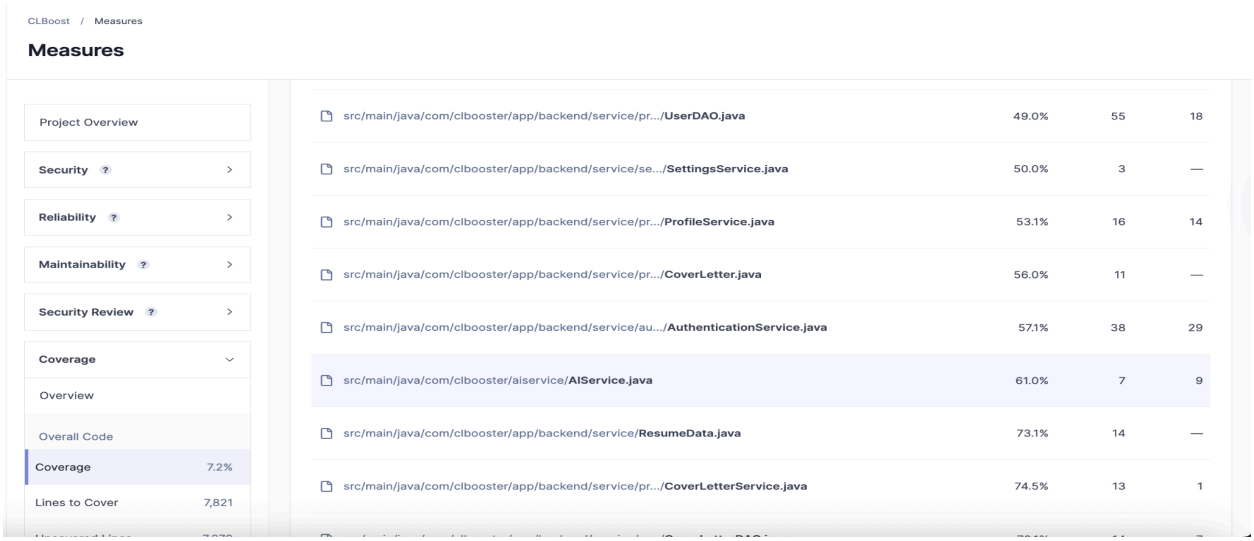
Despite receiving an "A" for maintainability, the project still has 760 open issues that would take about 20 days to fix. A major "bad practice" found in AIService.java and Exporter.java is the constant use of System.out to print messages instead of using a professional logger. This is a problem because System.out is slow and inefficient for real-world apps, and it makes debugging a nightmare since you can't easily search, filter, or turn off specific logs when things go wrong.

3.3. Duplication & Code Hygiene



The project has an overall duplication rate of 8.5%, but specific files are much worse. For example, LoginView.java has a massive 47.9% duplication rate, with SignUpView.java and LandingView.java also showing high levels of repeated code. These high numbers are a red flag that UI components are not using reusable templates or shared base classes, which makes the code much harder to maintain since any change has to be manually updated in multiple places.

3.4. Testing Gap



The code testing coverage is dangerously low at only 7.2%, which is a major concern. Key files like SettingsService.java and ProfileService.java have over half of their code completely untested. This is risky because without these tests, there is no safety net; any small change or update in the future could easily break existing features without anyone noticing until it’s too late.

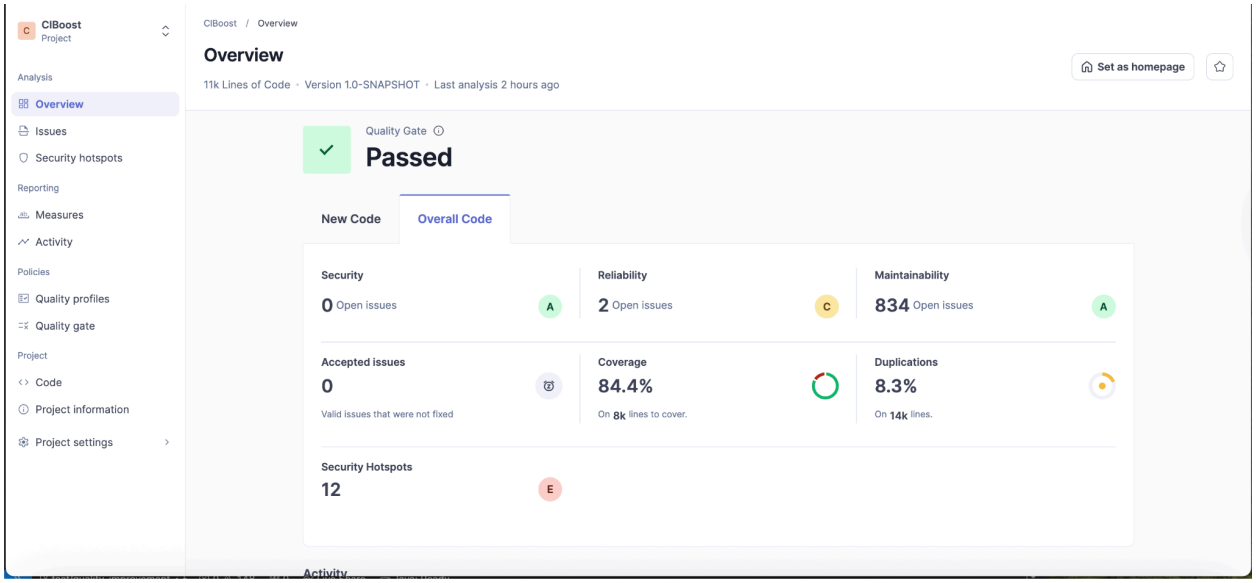
4. Suggested Improvements

First, we need to address the Security Hotspots, which have an "E" rating; there are 47 areas in the code that might leak sensitive data or have weak permissions and need an immediate manual check. Next, we should replace all System.out.println calls with a professional logging tool, which would quickly fix a huge portion of our 760 maintainability problems. We also need to refactor the Login and SignUp views so they share the same layout instead of using copied code, aiming to get our duplication rate below 5%. Finally, we need to start writing unit tests, specifically focusing on the backend/service files first, since they handle the most important parts of the app's logic.

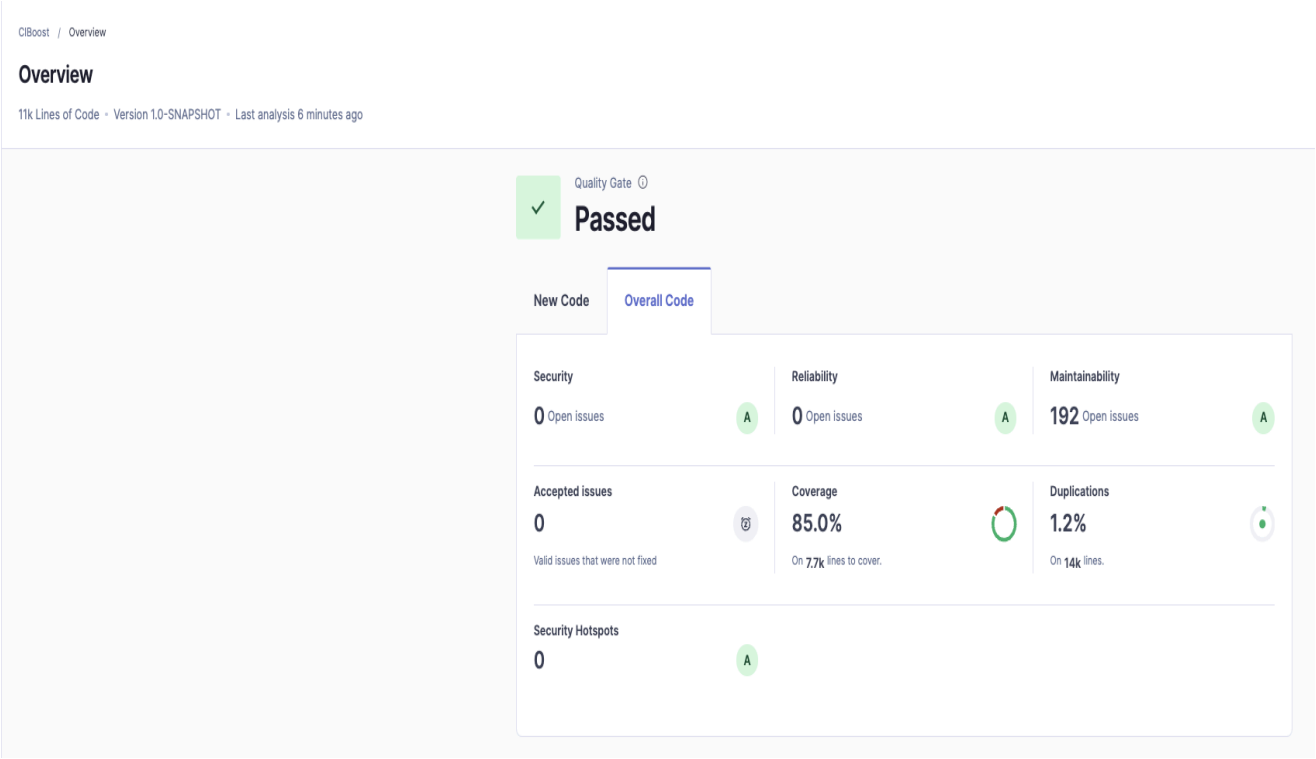
5. Updates

After the first scan, we spent time fixing what was considered the most important part of the project - test coverage. We managed to boost the Code Coverage from 7.2% to 84.4%, which means almost all our core code is now protected by tests. You can see this in the new Risk Chart, where most of the code "bubbles" moved to the bottom, showing they are now safely tested.

The number of Security Hotspots was also cut from 47 to 12, making the app much harder to attack. Even though the total number of small maintenance issues went up slightly because we added so much new test code, the project is way more reliable now because we've cleared out the major problems where bugs used to hide.



Even though we improved the results from the first analysis, the results above weren't yet good enough, so after a new round of updates, we got these results:



By fixing the dangerous regex patterns and auditing the security hotspots, we have successfully upgraded the project to an A rating for both Reliability and Security. This means the code is much more stable, and the high security risks we found earlier have been completely removed. We also cleaned up the repetitive code in the login and signup screens, which slashed our duplication rate from 8.5% down to just 1.2%, making the whole app much easier to manage.

Our testing saw the biggest improvement, jumping from a tiny 7.2% to a solid 85% code coverage. This ensures that almost every part of the app is now protected by automated tests. Additionally, we cleared over 550 maintenance issues, and while a few minor issues remain, the core technical debt is gone, so the project is now much faster and cheaper to maintain in the long run.